



机器学习导论

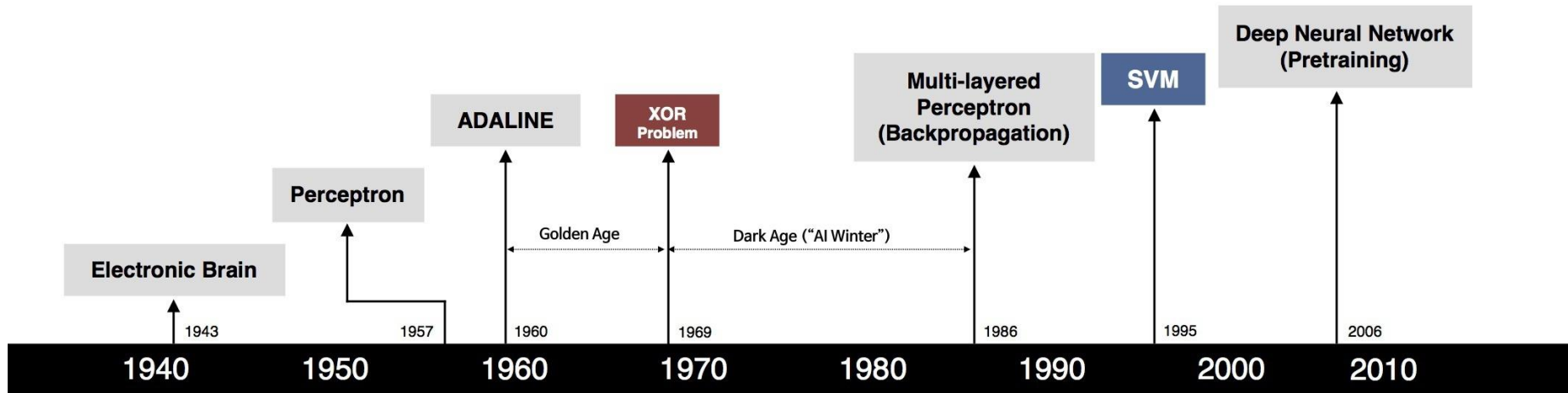
第5章 支持向量机

谢茂强

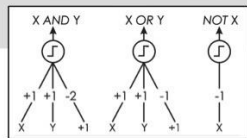
南开大学软件学院

本章主要内容来自周志华《机器学习》和吴恩达的Coursera网课

SVM在两轮神经网络间的辉煌



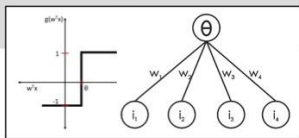
S. McCulloch - W. Pitts



- Adjustable Weights
- Weights are not Learned



F. Rosenblatt



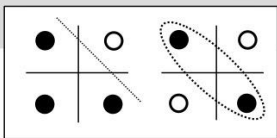
- Learnable Weights and Threshold



B. Widrow - M. Hoff



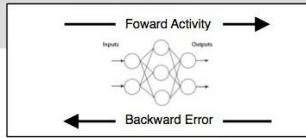
M. Minsky - S. Papert



- XOR Problem



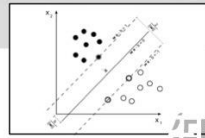
D. Rumelhart - G. Hinton - R. Williams



- Solution to nonlinearly separable problems
- Big computation, local optima and overfitting



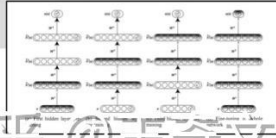
V. Vapnik - C. Cortes



- Limitations of learning prior knowledge
- Kernel function: Human Intervention



G. Hinton - S. Ruslan



- Hierarchical Feature Learning

休息一会儿

- 从90年代后期至2015年前后深度学习兴起之前, SVM被认为是机器学习当时最成功、表现最好的方法。

SVM的确与神经网络有密切联系: 若将隐层神经元数设置为训练样本数, 且每个训练样本对应一个神经元中心, 则以高斯径向基函数为激活函数的RBF网络(参见5.5.1节)恰与高斯核SVM的预测函数相同。

小故事: 统计学习理论之父弗拉基米尔·瓦普尼克

弗拉基米尔·瓦普尼克 (Vladimir N. Vapnik, 1936—) 是杰出的数学家、统计学家、计算机科学家。他出生于苏联, 1958年在乌兹别克国立大学获数学硕士学位, 1964年在莫斯科控制科学学院获统计学博士学位, 此后一直在该校工作并担任计算机系主任。1990年(苏联解体的前一年)他离开苏联来到新泽西州的美国电话电报公司贝尔实验室工作, 1995年发表了最初的SVM文章。当时神经网络正当红, 因此这篇文章被权威期刊 *Machine Learning* 要求以“支持向量网络”的名义发表。



实际上, 瓦普尼克在1963年就已提出了支持向量的概念, 1968年他与另一位苏联数学家 A. Chervonenkis 提出了以他们两人的姓氏命名的“VC维”, 1974年又提出了结构风险最小化原则, 使得统计学习理论在二十世纪七十年代就已成型。但这些工作主要是以俄文发表的, 直到瓦普尼克随着东欧剧变和苏联解体导致的苏联科学家移民潮来到美国, 这方面的研究才在西方学术界引起重视, 统计学习理论、支持向量机、核方法在二十世纪末大红大紫。

瓦普尼克2002年离开美国电话电报公司加入普林斯顿的 NEC 实验室, 2014年加盟脸书(Facebook)公司人工智能实验室。1995年之后他还在伦敦大学、哥伦比亚大学等校任教授。据说瓦普尼克在苏联根据一本字典自学了英语及其发音。他有一句名言被广为传诵: “*Nothing is more practical than a good theory.*”

- 从90年代后期至2010年前后深度学习兴起之前, SVM被认为是机器学习当时最成功、表现最好的方法。
- Vapnik(苏联人)于1963年提出支持向量概念
- Vapnik 和 Chervonenkis于1968年提出VC维, 是统计学习理论的基础。
- 目前普遍使用的SVM(支持向量机) 由 Cortes 和 Vapnik于1993年提出, 并在1995年发表。



Vladimir N. Vapnik(右) & Alexey Ya. Chervonenkis(左)

Vapnik, V.N. and Lerner, A.Y.. Recognition of patterns with help of generalized portraits. Automation and Remote Control. 24(6), pp.774-780. 1963.
Vapnik, V.N. and A. Y. Chervonenkis. The necessary and sufficient conditions for consistency in the empirical risk minimization method. Pattern Recognition and Image Analysis. 1 (3), 283-305. 1991.
Cortes, C. and Vapnik, V.. Support vector networks. Machine learning, 20(3), pp.273-297. 1995.

01.支持向量机(Support Vector Machine)

代价函数

结构风险+经验风险

结构风险: 最大化分割面的间隔

预测模型 (判决函数)

SVM代价函数的优化

代价函数的对偶形式

半正定二次规划

02.软间隔SVM

03.核函数(Kernel Function)

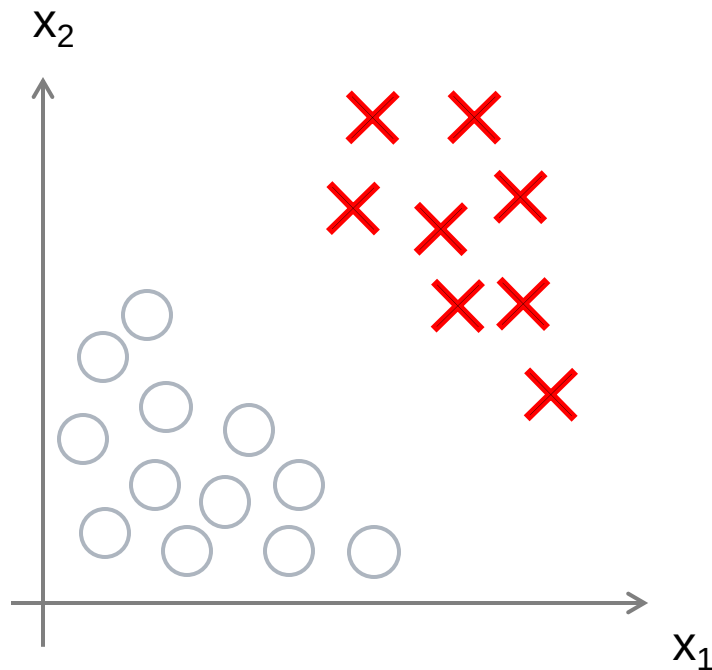
04.支持向量回归

1.1 线性可分问题 & 最大化间隔策略

$$\begin{aligned} \max_{\mathbf{w}, b} \quad & \frac{2}{\|\mathbf{w}\|} \\ \text{s.t.} \quad & y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \\ & i = 1, 2, \dots, m \end{aligned}$$

注意: $y_i \in \{-1, 1\}$

支持向量: 满足约束条件,
距离超平面最近的训练样
本



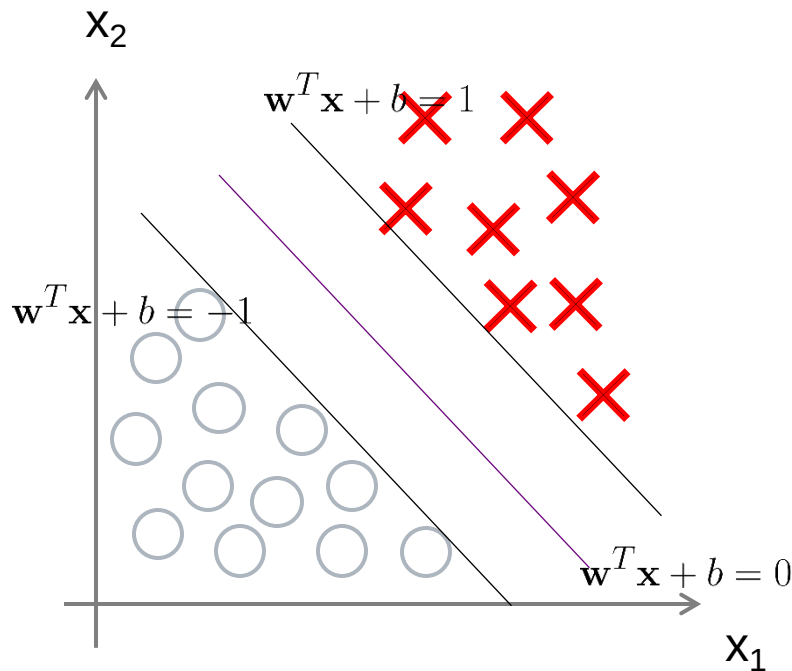
1.1 最大化间隔与结构风险

$$\frac{1}{2} \|\mathbf{w}\|^2 = \frac{1}{2} \sum_{j=1}^n \mathbf{w}^2$$

$\frac{2}{\|\mathbf{w}\|}$: 两类样本间的margin

点到直线距离: $\frac{|\mathbf{w}^T \mathbf{x} + b|}{\|\mathbf{w}\|}$

所以, $\mathbf{w}^T \mathbf{x} + b = 1$ 上的点到
 $\mathbf{w}^T \mathbf{x} + b = 0$ 是 $\frac{1}{\|\mathbf{w}\|}$ 。



1.1 上述最优化问题的拉格朗日函数

$$\begin{aligned} \max_{\mathbf{w}, b} \quad & \frac{2}{\|\mathbf{w}\|} \\ \text{s.t.} \quad & y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \\ & i = 1, 2, \dots, m \end{aligned}$$

$$\begin{aligned} L(\mathbf{w}, b, \alpha) = & \frac{1}{2} \|\mathbf{w}\|^2 + \sum_{i=1}^m \alpha_i (1 - y_i(\mathbf{w}^T \mathbf{x}_i + b)) \\ & \sum_{i=1}^m \alpha_i (1 - y_i(\mathbf{w}^T \mathbf{x}_i + b)) \end{aligned}$$

经验风险：度量预测模型与训练数据的误差

$$\frac{1}{2} \|\mathbf{w}\|^2 = \frac{1}{2} \sum_{j=1}^n \mathbf{w}^2$$

结构风险：描述模型的某些性质，以避免过拟合。(Maximizing the margin)

1.1 代价函数中的经验风险

$$\sum_{i=1}^m \alpha_i (1 - y_i (\mathbf{w}^T \mathbf{x}_i + b))$$

而 $\mathbf{w}^T \mathbf{x}_i + b$ 应该 ≥ 1 或 ≤ -1

最理想的是 y_i 和 $\mathbf{w}^T \mathbf{x}_i + b$ 输出相同，损失为0，

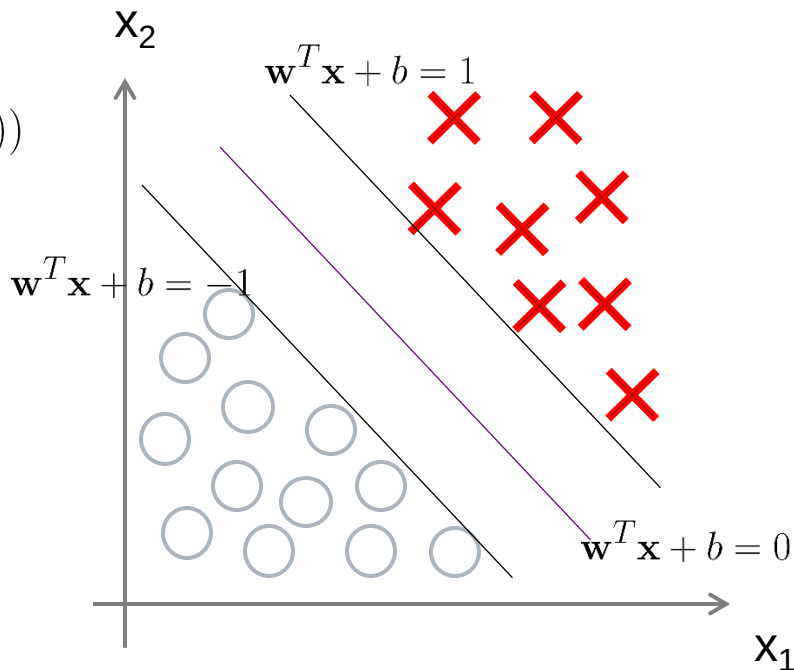
1.2 SVM的判决函数(之一)

当使用训练数据集优化代价函数

$$L(\mathbf{w}, b, \alpha) = \frac{1}{2} \|\mathbf{w}\|^2 + \sum_{i=1}^m \alpha_i (1 - y_i (\mathbf{w}^T \mathbf{x}_i + b))$$

得到模型参数 $\mathbf{w}, b$ 后，判决函数为：

$$f(\mathbf{x}_i; \mathbf{w}) = \begin{cases} 1, & \mathbf{w}^T \mathbf{x}_i + b \geq 1 \\ -1, & \mathbf{w}^T \mathbf{x}_i + b \leq -1 \end{cases}$$



01.支持向量机(Support Vector Machine)

代价函数

结构风险+经验风险

结构风险: 最大化分割面的间隔

预测模型 (判决函数)

SVM代价函数的优化

代价函数的对偶形式

半正定二次规划

02.软间隔SVM

03.核函数(Kernel Function)

04.支持向量回归

1.3 SVM的优化

最小化:
$$L(\mathbf{w}, b, \alpha) = \frac{1}{2} \|\mathbf{w}\|^2 + \sum_{i=1}^m \alpha_i (1 - y_i(\mathbf{w}^T \mathbf{x}_i + b))$$

可以通过优化其对偶形式（下式）加以求解：

$$\max_{\alpha} \{ \min_{\mathbf{w}, b} L(\mathbf{w}, b, \alpha) \}$$

- (1) 先通过求导法，求 $\min_{\mathbf{w}, b} L(\mathbf{w}, b, \alpha)$
- (2) 然后再求 $\min_{\mathbf{w}, b} L(\mathbf{w}, b, \alpha)$ 对 α 的极大。

1.3.1 通过求导法, 给定 α 求 $\min_{\mathbf{w}, b} L(\mathbf{w}, b, \alpha)$

给定 α 最小化: $L(\mathbf{w}, b, \alpha) = \frac{1}{2} \|\mathbf{w}\|^2 + \sum_{i=1}^m \alpha_i (1 - y_i(\mathbf{w}^T \mathbf{x}_i + b))$

- 分别对 $\mathbf{w}$ 和 b 偏导等于0

$$\frac{\partial}{\partial \mathbf{w}} L(\mathbf{w}, b, \alpha) = \mathbf{w} - \sum_{i=1}^m \alpha_i y_i \mathbf{x}_i = 0 \quad \text{得} \quad \mathbf{w} = \sum_{i=1}^m \alpha_i y_i \mathbf{x}_i$$

$$\frac{\partial}{\partial b} L(\mathbf{w}, b, \alpha) = - \sum_{i=1}^m \alpha_i y_i = 0 \quad \text{得} \quad 0 = \sum_{i=1}^m \alpha_i y_i$$

- 将 $\mathbf{w} = \sum_{i=1}^m \alpha_i y_i \mathbf{x}_i$ 带入最上式可得

$$L(\mathbf{w}, b, \alpha) = \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j - \sum_{i=1}^m \alpha_i y_i \left(\left(\sum_{j=1}^m \alpha_j y_j \mathbf{x}_j \right)^T \mathbf{x}_i + b \right) + \sum_{i=1}^m \alpha_i$$

给定 α 最小化:

$$L(\mathbf{w}, b, \alpha) = \frac{1}{2} \|\mathbf{w}\|^2 + \sum_{i=1}^m \alpha_i (1 - y_i(\mathbf{w}^T \mathbf{x}_i + b))$$

$$\begin{aligned} L(\mathbf{w}, b, \alpha) &= \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j - \sum_{i=1}^m \alpha_i y_i \left(\left(\sum_{j=1}^m \alpha_j y_j \mathbf{x}_j \right)^T \mathbf{x}_i + b \right) + \sum_{i=1}^m \alpha_i \\ &= \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \end{aligned}$$

(1) 先通过求导法, 求 $\min_{\mathbf{w}, b} L(\mathbf{w}, b, \alpha)$

$$\min_{\mathbf{w}, b} L(\mathbf{w}, b, \alpha) = \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$$

1.3.2 然后再求 $\min_{\mathbf{w}, b} L(\mathbf{w}, b, \alpha)$ 对 α 的极大。

$$\begin{aligned} \max_{\alpha} & \left(\sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \right) \\ \text{s.t.} & \sum_{i=1}^m \alpha_i y_i = 0 \\ & \alpha_i \geq 0, \quad i = 1, 2, \dots, m \end{aligned}$$

关于 α_i 的二次规划问题

- 1) 使用通用的二次规划(Quadratic Programming)求解
- 2) 使用专用的 SMO(Sequential Minimal Optimization)

求解关于 α_i 的二次规划问题所基于的定理(不要求)

定理 C.3 对原始问题 (C.1) ~ (C.3) 和对偶问题 (C.12) ~ (C.13), 假设函数 $f(\mathbf{x})$ 和 $c_i(\mathbf{x})$ 是凸函数, $h_j(\mathbf{x})$ 是仿射函数, 并且不等式约束 $c_i(\mathbf{x})$ 是严格可行的, 则 $\mathbf{x}^*$ 和 $\mathbf{a}^*, \beta^*$ 分别是原始问题和对偶问题的解的充分必要条件是 $\mathbf{x}^*, \mathbf{a}^*, \beta^*$ 满足下面的 Karush-Kuhn-Tucker (KKT) 条件:

$$\nabla_{\mathbf{x}} L(\mathbf{x}^*, \alpha^*, \beta^*) = 0$$

$$\nabla_{\alpha} L(\mathbf{x}^*, \alpha^*, \beta^*) = 0$$

$$\nabla_{\beta} L(\mathbf{x}^*, \alpha^*, \beta^*) = 0$$

$$\alpha_i^* c_i(\mathbf{x}^*) = 0, \quad i = 1, 2, \dots, k$$

$$c_i(\mathbf{x}^*) \leq 0, \quad i = 1, 2, \dots, k$$

$$\alpha_i^* \geq 0, \quad i = 1, 2, \dots, k$$

$$h_j(\mathbf{x}^*) = 0, \quad i = 1, 2, \dots, k$$

由 $\mathbf{w} = \sum_{i=1}^m \alpha_i y_i \mathbf{x}_i$ 和 $\mathbf{w}^* > 0$

可知存在一个 $\alpha_j^* > 0$

$$\alpha_i (y_i (\mathbf{w}^T \mathbf{x}_i + b) - 1) = 0.$$

得: $y_j (\mathbf{w}^{*T} \mathbf{x}_j + b^*) - 1 = 0$

或: $y_j (\mathbf{w}^{*T} \mathbf{x}_j + b^*) = y_j^2$

用于求解关于 α 的二次规划问题的KKT条件

KKT(Karush-Kuhn-Tucker)条件是对应二次规划问题的充分必要条件。

$$\begin{cases} \alpha_i \geq 0; \\ y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 \geq 0; \\ \alpha_i(y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1) = 0. \end{cases}$$

若 $\alpha_i > 0$, 则 $y_i(\mathbf{w}^T \mathbf{x} + b) = 1$, 对应样本位于最大间隔边界上, 是一个支持向量。

1.3.3 根据(1)和(2)的结果求解 w^* 和 b^*

可得：

$$w^* = \sum_{i=1}^m \alpha_i^* y_i x_i$$

$$b^* = y_j - \sum_{i=1}^m \alpha_i^* y_i x_i^T x_j$$

注：选择一个 $\alpha_j^* > 0$ 对应的 (x_j, y_j) 来计算 b^*

$$\begin{aligned} f(\mathbf{x}) &= \text{sign}(\mathbf{w}^T \mathbf{x} + b) \\ &= \text{sign}\left(\sum_{i=1}^m \alpha_i y_i \mathbf{x}_i^T \mathbf{x} + b\right) \end{aligned}$$

2. 软间隔SVM

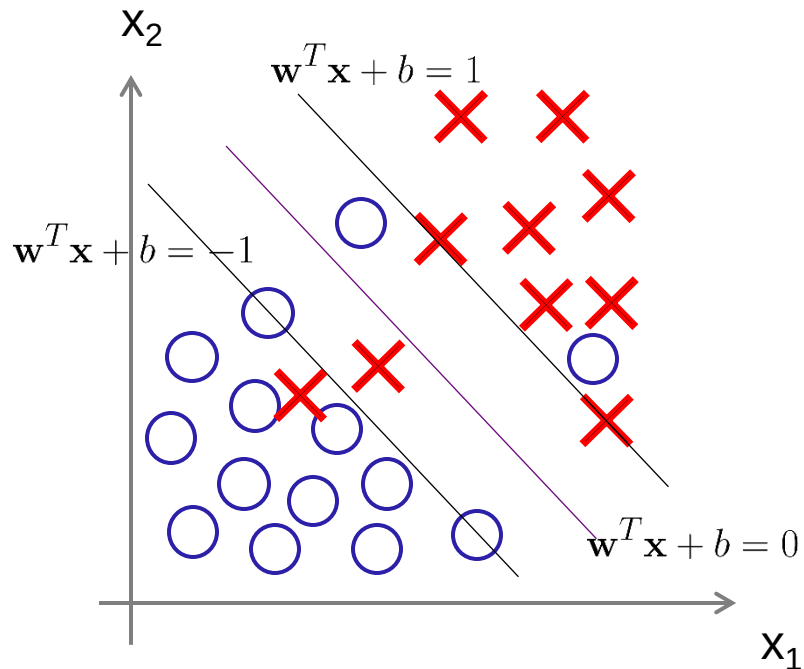
$$\min_{\mathbf{w}, b, \xi_i} \frac{1}{2} \|\mathbf{w}\|^2$$

$$\begin{aligned} \text{s.t. } & y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \\ & i = 1, 2, \dots, m \end{aligned}$$



$$\min_{\mathbf{w}, b, \xi_i} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i$$

$$\begin{aligned} \text{s.t. } & y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i \\ & \xi_i \geq 0, i = 1, 2, \dots, m \end{aligned}$$

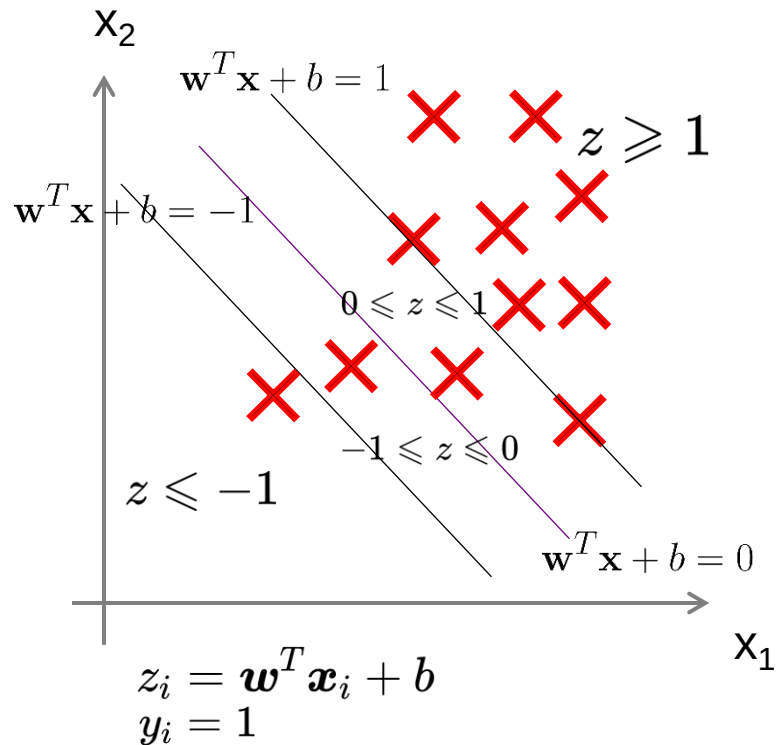
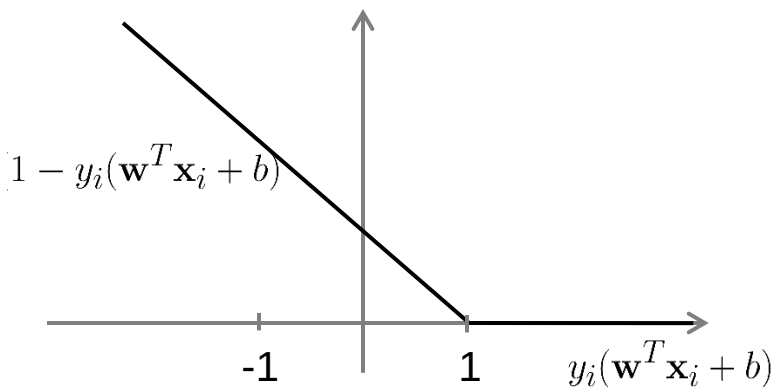


2. 用于软间隔SVM的Hinge损失

$$\begin{cases} \xi_i = 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b) \\ \xi_i \geq 0 \end{cases}$$



$$\max(0, 1 - y_i(\mathbf{w}^T \mathbf{x} + b))$$



2. 软间隔SVM的优化

对于线性不完全可分的情况，引入软间隔的SVM，即允许某些样本不满足约束：

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1$$



$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, i = 1, 2, \dots, m$$



$$\min_{\mathbf{w}, b, \alpha, \xi, \mu} \frac{1}{2} \|\mathbf{w}\|^2 + \sum_{i=1}^m \alpha_i (1 - \xi_i - y_i(\mathbf{w}^T \mathbf{x}_i + b)) + C \sum_{i=1}^m \xi_i - \sum_{i=1}^m \mu_i \xi_i$$

$$\begin{aligned} \min_{\mathbf{w}, b, \xi_i} & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i \\ \text{s.t.} & y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i \\ & \xi_i \geq 0, i = 1, 2, \dots, m \end{aligned}$$

$$\max_{\alpha} \min_{w, b} \mathcal{L}(\mathbf{w}, b, \alpha) = \max_{\alpha} \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$$

$$s.t. \sum_{i=1}^m \alpha_i y_i = 0$$



$$\max_{\alpha} \min_{\mathbf{w}, b, \xi, \mu} L(\mathbf{w}, b, \alpha, \xi, \mu) = \max_{\alpha} \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$$

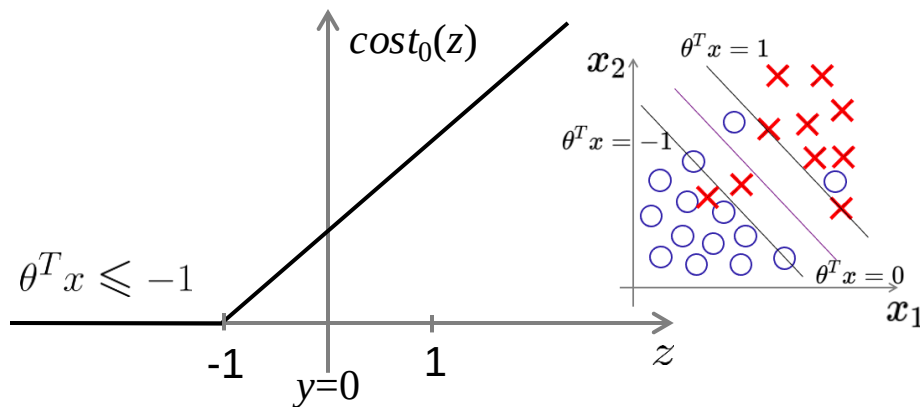
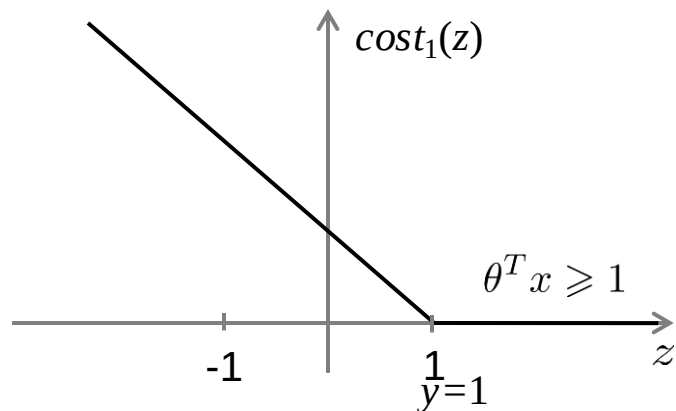
$$s.t. \sum_{i=1}^m \alpha_i y_i = 0 \quad , \quad 0 \leq \alpha_i \leq C, i = 1, 2, \dots, m$$

$$\left\{ \begin{array}{l} \alpha_i \geq 0; \\ y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 \geq 0; \\ \alpha_i(y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1) = 0. \end{array} \right. \rightarrow \left\{ \begin{array}{l} \alpha_i \geq 0, \mu_i \geq 0, \\ y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 + \xi_i \geq 0, \\ \alpha_i(y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 + \xi_i) = 0, \\ \xi_i \geq 0, \mu_i \xi_i = 0. \end{array} \right.$$

2. 吴恩达课程中的软间隔SVM的损失函数

$$y^{(i)} = 1 \text{ or } 0$$

$$\min_{\theta} \left\{ C \sum_{i=1}^m \left[y^{(i)} \text{cost}_1(\theta^T x^{(i)}) + (1 - y^{(i)}) \text{cost}_0(\theta^T x^{(i)}) \right] + \frac{1}{2} \sum_{j=1}^n \theta_j^2 \right\}$$



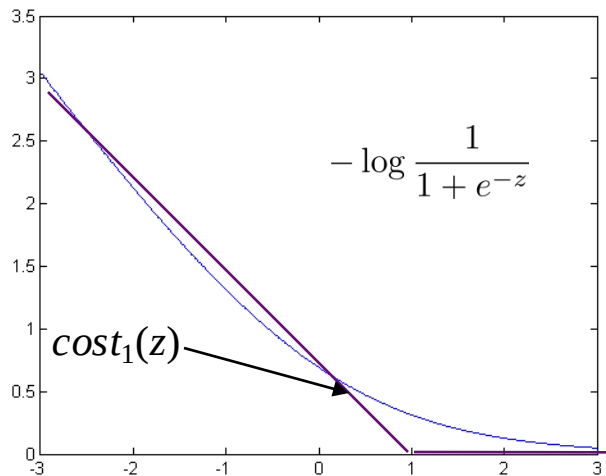
If $y = 1$, we want $\theta^T x \geq 1$ (not just ≥ 0)

If $y = 0$, we want $\theta^T x \leq -1$ (not just < 0)

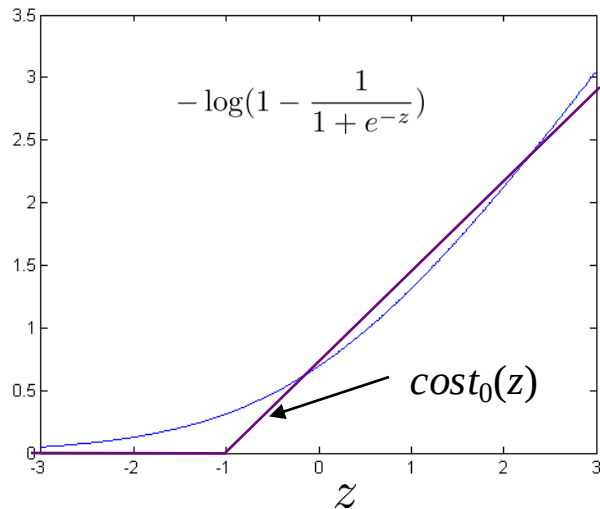
对比logistic回归和软间隔SVM的损失函数

Logistic回归: $-y \log \frac{1}{1 + e^{-\theta^T x}} - (1 - y) \log(1 - \frac{1}{1 + e^{-\theta^T x}})$

当 $y = 1$, 期望 $\theta^T x \gg 0$



当 $y = 0$, 期望 $\theta^T x \ll 0$



【注】：此处SVM的 $y_i \in \{0, 1\}$

调整C对判决模型的影响:

$$\min_{\theta} C \sum_{i=1}^m \left[y^{(i)} \text{cost}_1(\theta^T x^{(i)}) + (1 - y^{(i)}) \text{cost}_0(\theta^T x^{(i)}) \right] + \frac{1}{2} \sum_{j=1}^n \theta_j^2$$

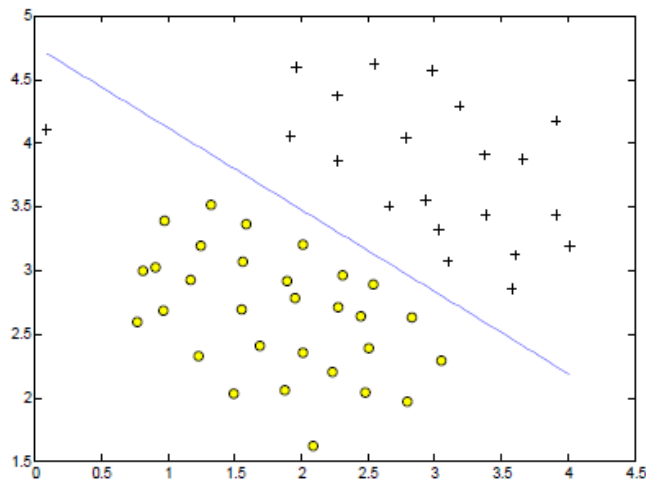


Figure 2: SVM Decision Boundary with $C = 1$ (Example Dataset 1)

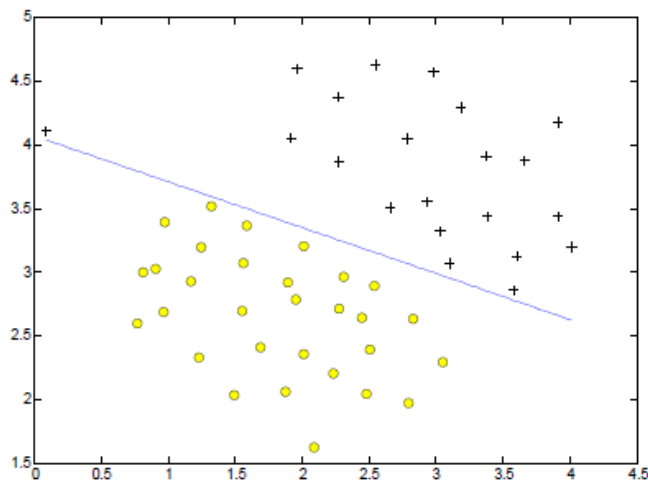


Figure 3: SVM Decision Boundary with $C = 100$ (Example Dataset 1)

详见吴恩达网课ex6中在数据集1上的实验结果

01.支持向量机(Support Vector Machine)

代价函数

结构风险+经验风险

结构风险: 最大化分割面的间隔

预测模型 (判决函数)

SVM代价函数的优化

代价函数的对偶形式

半正定二次规划

02.软间隔SVM

03.核函数(Kernel Function)

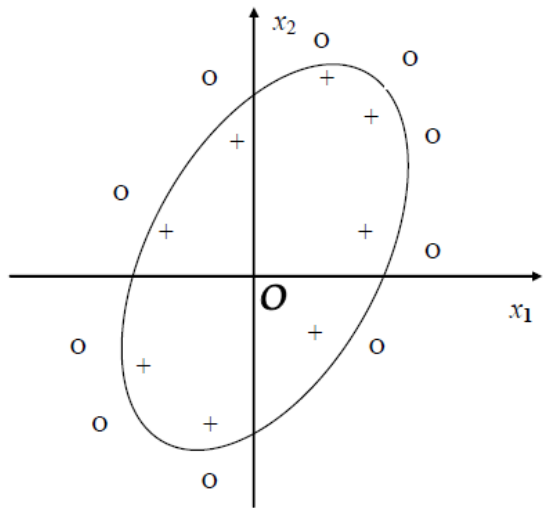
04.支持向量回归

3. 通过空间变换将非线性可分问题转变为线性可分问题

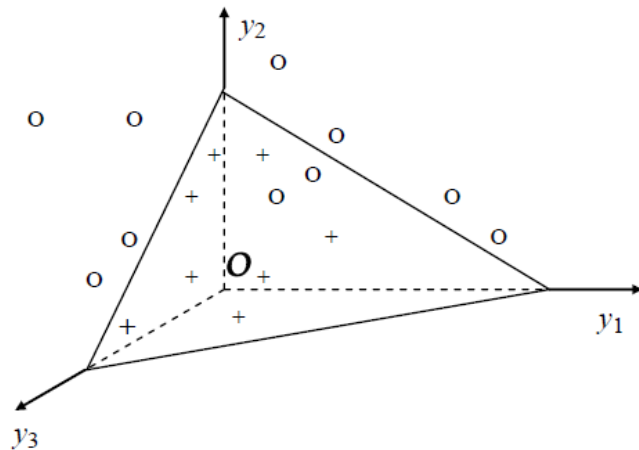
$$w_1 x_1^2 + w_2 x_2^2 + \sqrt{2} w_3 x_1 x_2 + b = 0$$

$$\phi(\vec{x}) = \phi(x_1, x_2) = (y_1, y_2, y_3) = (x_1^2, x_2^2, \sqrt{2} x_1 x_2)$$

$$w_1 y_1 + w_2 y_2 + w_3 y_3 + b = 0$$



(a) 在 x_1 和 x_2 张成的空间中



(b) 在 $x_1^2, x_2^2, \sqrt{2} x_1 x_2$ 张成的空间中

3. 核函数：考虑空间变换前后的相似度量变换

核函数 (kernel function) $K(\bar{\mathbf{x}}, \bar{\mathbf{y}})$ 是一个内积函数, 它对于所有 $\mathbf{x}, \mathbf{y} \in X \subseteq \mathbf{R}^d$ 满足:

$$K(\mathbf{x}, \mathbf{y}) = \langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle$$

其中, ϕ 是从 X 到 (内积) 特征空间 F 的一个映射。

$$\phi: \mathbf{x} \mapsto \phi(\mathbf{x}) \in F$$

表 6.1 常用核函数

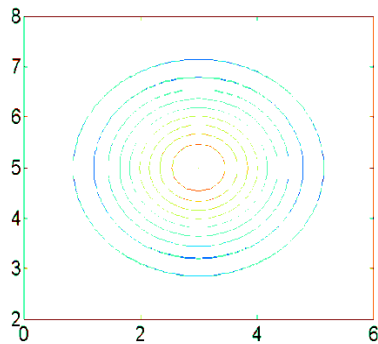
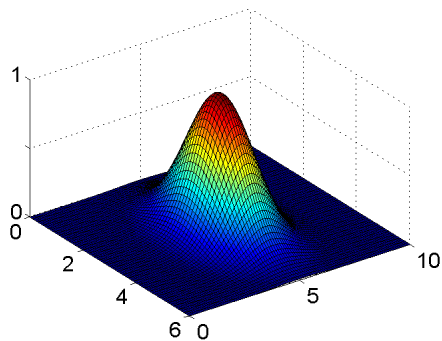
名称	表达式	参数
线性核	$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$	
多项式核	$\kappa(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^T \mathbf{x}_j)^d$	$d \geq 1$ 为多项式的次数
高斯核	$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\ \mathbf{x}_i - \mathbf{x}_j\ ^2}{2\sigma^2}\right)$	$\sigma > 0$ 为高斯核的带宽(width)
拉普拉斯核	$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\ \mathbf{x}_i - \mathbf{x}_j\ }{\sigma}\right)$	$\sigma > 0$
Sigmoid 核	$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\beta \mathbf{x}_i^T \mathbf{x}_j + \theta)$	$\tanh$ 为双曲正切函数, $\beta > 0, \theta < 0$

$d = 1$ 时退化为线性核.
高斯核亦称 RBF 核.

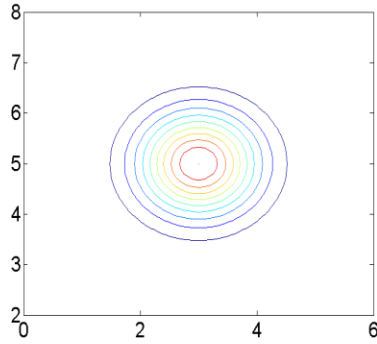
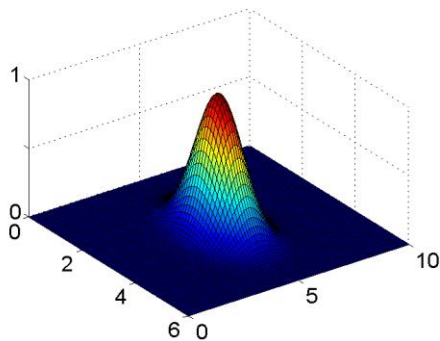
3. 最常用的核函数：高斯核函数

$$l^{(1)} = \begin{bmatrix} 3 \\ 5 \end{bmatrix}, \quad f_1 = \exp \left(-\frac{\|x - l^{(1)}\|^2}{2\sigma^2} \right)$$

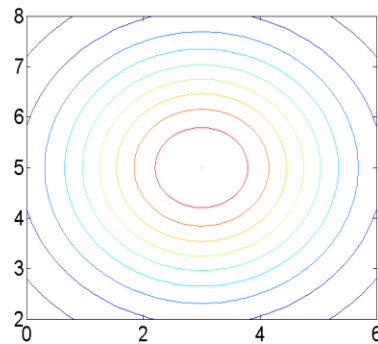
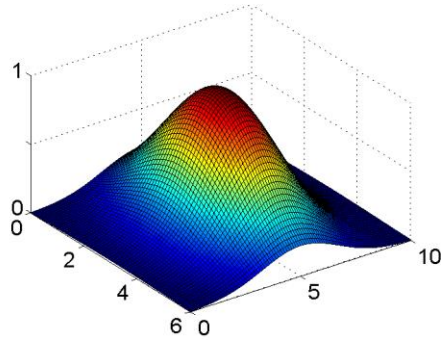
$$\sigma^2 = 1$$



$$\sigma^2 = 0.5$$



$$\sigma^2 = 3$$



$$\max_{\alpha} \min_{w, b} \mathcal{L}(\mathbf{w}, b, \alpha) = \max_{\alpha} \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$$

$$s.t. \sum_{i=1}^m \alpha_i y_i = 0$$


$$\max_{\alpha} \min_{\mathbf{w}, b, \xi, \mu} L(\mathbf{w}, b, \alpha, \xi, \mu) = \max_{\alpha} \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$$

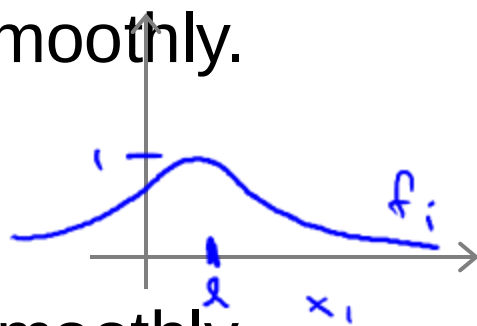
$$s.t. \sum_{i=1}^m \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C, i = 1, 2, \dots, m$$

软间隔SVM的超参数与核函数参数

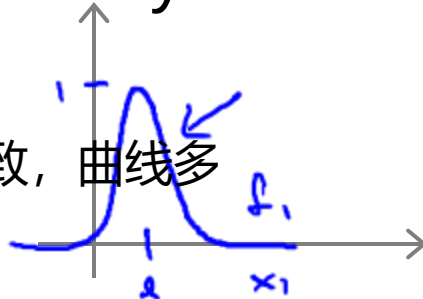
$C (= \frac{1}{\lambda}) \Rightarrow$ Large C : Lower bias, high variance. (small λ)
 $\rightarrow$ Small C : Higher bias, low variance. (large λ)

σ^2 : Large σ^2 : Features f_i vary more smoothly.
 $\rightarrow$ Higher bias, lower variance.

$$\exp\left(-\frac{\|x - x^{(i)}\|^2}{2\sigma^2}\right)$$



Small σ^2 : Features f_i vary less smoothly.
 Lower bias, higher variance.



Lower bias, higher variance. Large C

Higher bias, lower variance Small C

Small σ^2 分类面细致, 曲线多

Large σ^2

调整 σ^2 对判决模型的影响:

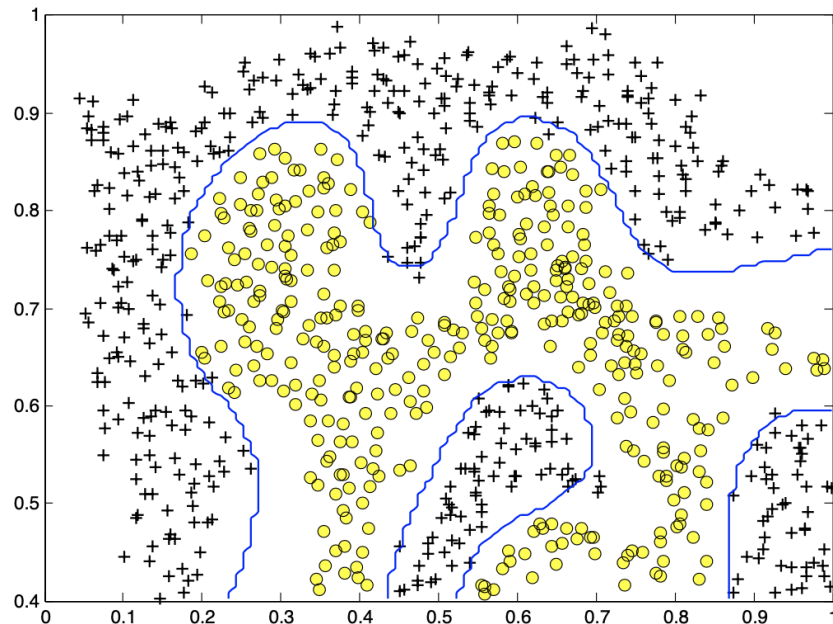


Figure 5: SVM (Gaussian Kernel) Decision Boundary (Example Dataset 2)

详见吴恩达网课ex6中在数据集2上的实验结果

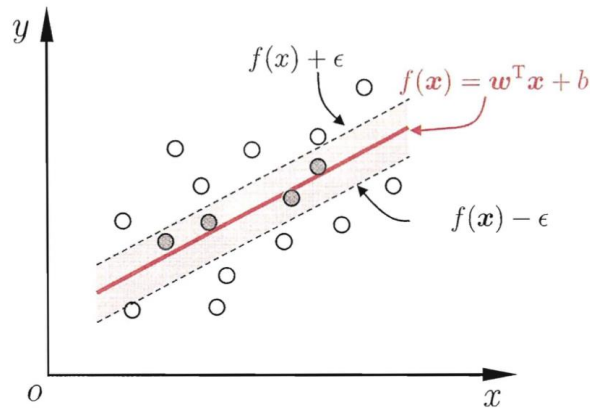
4. 支持向量回归-SVR

传统回归： $f(x)$ 与 y 完全相等时损失为0；

SVR： $f(x)$ 与 y 的差异大于 ϵ 时才计算损失。

落入 ϵ 间隔带中不计算
损失。

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \max(0, |f(x_i) - y| - \epsilon)$$



4. SVR的代价函数

$$\max_{\theta} = \frac{2}{\|\theta\|} \quad \text{或} \quad \min_{\theta} \frac{1}{2} \|\theta\|^2$$

$$s.t. \quad y_i(\Theta^T \mathbf{x}_i + b) \geq 1, \quad i = 1, 2, \dots, m$$

- 拉格朗日乘子法可得如下损失函数：

$$L(\mathbf{w}, b, \alpha) = \frac{1}{2} \|\theta\|^2 + \sum_{i=1}^m \alpha_i (1 - y_i(\mathbf{w}^T \mathbf{x}_i + b))$$

小 测

$$1 \quad \min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$$

$$s.t. \quad 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b) \leq 0$$

A)这是根据求解到的alpha最优解后所得到的w的最优解

$$2 \quad \min_{\mathbf{w}, b, \alpha} \frac{1}{2} \|\mathbf{w}\|^2 + \sum_{i=1}^m \alpha_i (1 - y_i(\mathbf{w}^T \mathbf{x}_i + b))$$

B)原始的代价函数

$$3 \quad \max_{\alpha} \min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + \sum_{i=1}^m \alpha_i (1 - y_i(\mathbf{w}^T \mathbf{x}_i + b))$$

C)使用拉格朗日乘子法将原始代价函数及约束转换为单一的优化问题

$$4 \quad \max_{\alpha} \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$$

D)这是原始代价函数的最大最小对偶

$$5 \quad \mathbf{w}^* = \sum_{i=1}^m \alpha_i^* y_i \mathbf{x}_i$$

E)这是最大最小对偶第1步的结果, 该问题是关于alpha的二次规划问题, 其最优解满足KKT条件

原题：让直线 $x + y = 1$ 与椭圆 $x^2 + 2y^2 = C$ (C 为常数) 相切，求 C 取值。

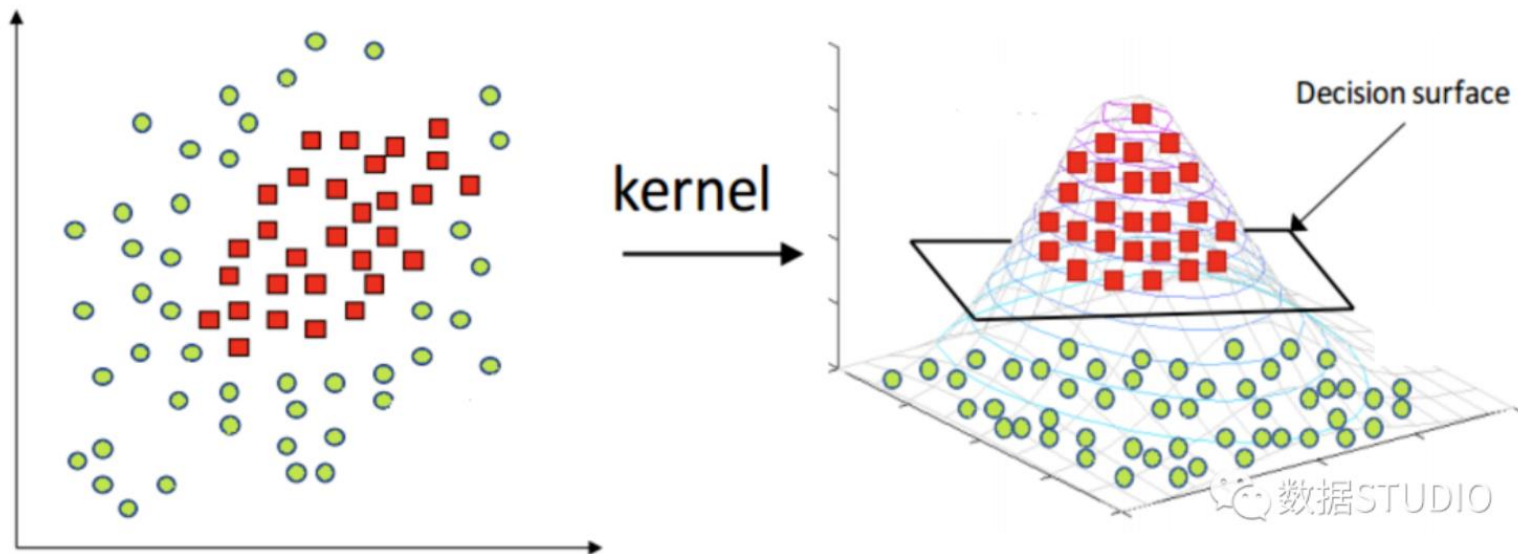
题目：请使用拉格朗日乘子法 (Lagrange Multiplier Method) 求解。

提示：可将其转换为如下优化问题：

$$\begin{aligned} \min_{x,y} & (x^2 + 2y^2) \\ s.t. & x + y = 1 \end{aligned}$$

4.1 核函数

如下图所示，核技巧的基本思路分为两步：使用一个变换将原空间的数据映射到新空间（例如更高维甚至无穷维的空间）；然后在新空间里用线性方法从训练数据中学习得到模型。



怎样映射到特征空间？

先来看看核函数的定义，

设 $\mathcal{X}$ 是输入空间(欧式空间 R^n 的子集或离散集合)，又设 $\mathcal{H}$ 是特征空间(希尔伯特空间)，如果存在一个 $\mathcal{X}$ 到 $\mathcal{H}$ 的映射 $\phi(x) : \mathcal{X} \rightarrow \mathcal{H}$ 使得对所有 $x, z \in \mathcal{X}$ ，函数 $K(x, z)$ 满足条件 $K(x, z) = \phi(x) \cdot \phi(z)$ 则称 $K(x, z)$ 为核函数， $\phi(x)$ 为映射函数，式中 $\phi(x) \cdot \phi(z)$ 为 $\phi(x)$ 和 $\phi(z)$ 的内积。

通常，直接计算 $K(x, z)$ 比较容易而通过 $\phi(x)$ 和 $\phi(z)$ 计算 $K(x, z)$ 并不容易。而幸运的是，在线性支持向量机的对偶问题中，无论是目标函数还是决策函数都只涉及到输入样本与样本之间的内积，因此我们不需要显式地定义映射 $\phi(x)$ 是什么而只需事先定义核函数 $K(x, z)$ 即可。也就是说，在核函数 $K(x, z)$ 给定的情况下，可以利用解线性问题的方法求解非线性问题的支持向量机，此过程是隐式地在特征空间中进行的。

4.2 正定核

由上面的介绍可知，我们只需要定义核函数就可以了。但是如何不通过映射 $\phi(x)$ 判断给定的一个函数 $K(x, z)$ 是不是核函数呢？或者说， $K(x, z)$ 需要满足什么条件才是一个核函数。

通常所说的核函数就是正定核函数，下面不加证明的给出正定核的充要条件，具体证明略显复杂，有兴趣的可以参考《统计学习方法》。

设 $\mathcal{X} \subset R^n$ ， $K(x, z)$ 是定义在 $\mathcal{X} \times \mathcal{X}$ 上的对称函数，如果对任意的 $x_i \in \mathcal{X}, i = 1, 2, \dots, m$ ， $K(x, z)$ 对应的Gram矩阵 $K = [K(x_i, x_j)]_{m \times m}$ 是半正定矩阵，则 $K(x, z)$ 是正定核。

虽然有了上述定义，但是实际应用时验证 $K(x, z)$ 是否是正定核依然不容易，因此在实际问题中一般使用已有的核函数，下面给出一些常用的核函数。

- 多项式核函数(polynomial kernel function)

$$K(x, z) = (x \cdot z + 1)^p \quad (4.2.1)$$

- 高斯核函数(Gaussian kernel function)

$$K(x, z) = \exp\left(-\frac{\|x - z\|^2}{2\sigma^2}\right) \quad (4.2.2)$$

4.3 非线性支持向量机

如前4.1、4.2所述，利用核技巧可以很简单地把线性支持向量机扩展到非线性支持向量机，只需将线性支持向量机中的内积换成核函数即可。下面简述非线性支持向量机学习算法。

- 首先选取适当的核函数 $K(x, z)$ 和适当的参数 C ，构造最优化问题

$$\begin{aligned} \min_{\alpha} \quad & \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(X_i, X_j) - \sum_{i=1}^n \alpha_i \\ \text{s.t.} \quad & \sum_{i=1}^n \alpha_i y_i = 0 \\ & 0 \leq \alpha_i \leq C, i = 1, 2, \dots, n. \end{aligned} \quad (4.3.1)$$

- 再利用现成的二次规划问题求解算法或者SMO算法求得最优解 $\hat{\alpha}$ 。
- 选择 $\hat{\alpha}$ 的一个满足 $0 < \hat{\alpha}_j < C$ 的分量 $\hat{\alpha}_j$ ，计算

$$\hat{b} = y_j - \sum_{i \in SV} \hat{\alpha}_i y_i K(X_j, X_i) \quad (4.3.2)$$

- 构造决策函数：

$$f(x) = \text{sign}\left(\sum_{i \in SV} \hat{\alpha}_i y_i K(X_j, X_i) + \hat{b}\right) \quad (4.3.3)$$

— 支持向量机的优点是:

1. 由于SVM是一个凸优化问题, 所以求得的解一定是全局最优而不是局部最优。
2. 不仅适用于线性线性问题还适用于非线性问题(用核技巧)。
3. 拥有高维样本空间的数据也能用SVM, 这是因为数据集的复杂度只取决于支持向量而不是数据集的维度, 这在某种意义上避免了“维数灾难”。
4. 理论基础比较完善(例如神经网络就更像一个黑盒子)。

支持向量机的缺点是:

1. 二次规划问题求解将涉及 m 阶矩阵的计算(m 为样本的个数), 因此SVM不适用于超大数据集。(SMO算法可以缓解这个问题)
2. 只适用于二分类问题。(SVM的推广SVR也适用于回归问题; 可以通过多个SVM的组合来解决多分类问题)

Primal problem

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum_{i=1}^N \xi_i \\ \text{s.t.} \quad & y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \quad i = 1, \dots, N \\ & \xi_i \geq 0, \quad i = 1, \dots, N \end{aligned}$$

Lagrange dual problem

$$\begin{aligned} \min_{\alpha} \quad & \frac{1}{2} \alpha^T \mathbf{Q} \alpha - \mathbf{1}^T \alpha \\ \text{s.t.} \quad & \mathbf{0} \preceq \alpha \preceq C \cdot \mathbf{1} \\ & \alpha^T \mathbf{y} = 0 \end{aligned}$$

支持向量机原理详解(二): 拉格朗日对偶函数, SVM的对偶问题

scikit-learn

Machine Learning in Python

[Getting Started](#)
[Release Highlights for 1.4](#)
[GitHub](#)

- Simple and efficient tools for predictive data analysis
- Accessible to everybody, and reusable in various contexts
- Built on NumPy, SciPy, and matplotlib
- Open source, commercially usable - BSD license

Classification

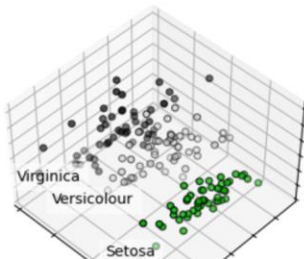
Identifying which category an object belongs

Dimensionality reduction

Reducing the number of random variables to consider.

Applications: Visualization, Increased efficiency

Algorithms: PCA, feature selection, non-negative matrix factorization, and more...



Regression

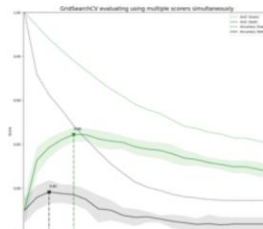
Predicting a continuous-valued attribute as-

Model selection

Comparing, validating and choosing parameters and models.

Applications: Improved accuracy via parameter tuning

Algorithms: grid search, cross validation, metrics, and more...



Clustering

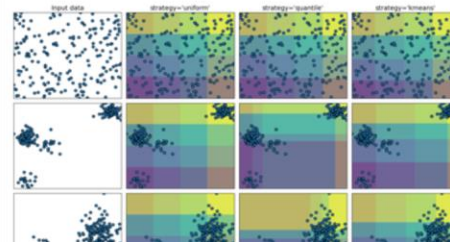
Automatic grouping of similar objects into

Preprocessing

Feature extraction and normalization.

Applications: Transforming input data such as text for use with machine learning algorithms.

Algorithms: preprocessing, feature extraction, and more...



```
from sklearn import svm
```

```
clf = svm.SVC(kernel=kernel, gamma=10)  
clf.fit(X_train, y_train)
```

```
clf.predict(X_test)
```

